

# pandas.DataFrame.mul

Calculate the revenue for each product on each day

Get Multiplication of dataframe and other, element-wise (binary operator mul).

units\_sold:

|     | Prod_A | Prod_B | Prod_C |
|-----|--------|--------|--------|
| Mon | 10     | 5      | 8      |
| Tue | 20     | 7      | 10     |
| Wed | 15     | 12     | 6      |
| Thu | 13     | 8      | 7      |
| Fri | 18     | 6      | 9      |

prices:

```
Prod_A    100  
Prod_B    300  
Prod_C    200  
dtype: int64
```



Revenue for each product on each day:

|     | Prod_A | Prod_B | Prod_C |
|-----|--------|--------|--------|
| Mon | 1000   | 1500   | 1600   |
| Tue | 2000   | 2100   | 2000   |
| Wed | 1500   | 3600   | 1200   |
| Thu | 1300   | 2400   | 1400   |
| Fri | 1800   | 1800   | 1800   |

Code Snippet:

```
units_sold.mul(prices, axis=1)
```

## Contents

What Makes This Ebook Unique

Tips for Effective Use of This Ebook

Introduction

The "other" Parameter

The "axis" Parameter

The "level" Parameter

The "fill\_value" Parameter

Author Biography

Other Ebooks

Contacts and Social Media

Sources & References

Thank You!

\*\*\*\*\*

# 1st Edition

## Why This E-book?

"The aim of this ebook is to give you the 'aha' moment right away at the start of learning a new concept."

- Practical step By Step Guide With Simple Examples
- Visual Illustrations and Interactive
- Simple Datasets
- Comprehensive Coverage (pandas Documentation used as reference)

## Tips for Effective Use of This Ebook

**Use Examples First:** If you don't understand the text, go straight to the examples—they're self-explanatory. After reviewing them, return to the text to grasp the practical concepts.

**Practice with Datasets:** Copy the datasets provided and practice using them to reinforce your understanding.

Click on the blue links to go directly to the section you want to learn about.

## Introduction

```
#import libraries
import numpy as np
import pandas as pd
```

Get Multiplication of dataframe and other, element-wise (binary operator mul).

units\_sold:

|     | Prod_A | Prod_B | Prod_C |
|-----|--------|--------|--------|
| Mon | 10     | 5      | 8      |
| Tue | 20     | 7      | 10     |
| Wed | 15     | 12     | 6      |
| Thu | 13     | 8      | 7      |
| Fri | 18     | 6      | 9      |

prices:

```
Prod_A 100
Prod_B 300
Prod_C 200
dtype: int64
```



Revenue for each product on each day:

|     | Prod_A | Prod_B | Prod_C |
|-----|--------|--------|--------|
| Mon | 1000   | 1500   | 1600   |
| Tue | 2000   | 2100   | 2000   |
| Wed | 1500   | 3600   | 1200   |
| Thu | 1300   | 2400   | 1400   |
| Fri | 1800   | 1800   | 1800   |

Code Snippet:  
units\_sold.mul(prices, axis=1)

## Syntax

`DataFrame.mul(other, axis='columns', level=None, fill_value=None)`

The `other` parameter

When `other` accepts a `scalar`:

A scalar is a single number.

Example: Multiplying all elements of a DataFrame by 10.

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |

Code Snippet:

`df.mul(10)`

Output:

|   | A  | B  |
|---|----|----|
| 0 | 10 | 20 |
| 1 | 30 | 40 |

Code Snippet:

```
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

`df`

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |

When `other` accepts a `series`:

Multiplying each column by a

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |

series:

A 10  
B 20

Output:

|   | A  | B  |
|---|----|----|
| 0 | 10 | 40 |

Code Snippet:

different number

|   | A | B |
|---|---|---|
| 1 | 3 | 4 |

dtype: int64



|   | A  | B  |
|---|----|----|
| 1 | 30 | 80 |

df.mul(other=series)

Code Snippet:

```
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

df

```
series = pd.Series([10, 20], index=['A', 'B'])
```

series

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |



A 10

B 20

dtype: int64

When **other** accepts a **Dict**:

A dictionary where keys match DataFrame columns.

We Multiply each column by a number specified in the dictionary.

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |

dictionary:

{'A': 10, 'B': 20}



Output:

|   | A  | B  |
|---|----|----|
| 0 | 10 | 40 |
| 1 | 30 | 80 |

Code Snippet:

df.mul(other = dictionary)

Code Snippet:

```
df=pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

df

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |



```
dictionary = {'A': 10, 'B': 20}
```

```
dictionary
```

```
{'A': 10, 'B': 20}
```

When `other` accepts a `DataFrame`:

A dictionary where keys match `DataFrame` columns.

A `DataFrame` with the same shape or compatible shape.

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |

df2:

|   | A  | B  |
|---|----|----|
| 0 | 10 | 20 |
| 1 | 30 | 40 |



Output:

|   | A  | B   |
|---|----|-----|
| 0 | 10 | 40  |
| 1 | 90 | 160 |

Code Snippet:

```
df1.mul(df2)
```

Code Snippets:

```
df1 = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

```
df1
```

```
df2 = pd.DataFrame([[10, 20], [30, 40]], columns=['A', 'B'])
```

```
df2
```

df1:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |



df2:

|   | A  | B  |
|---|----|----|
| 0 | 10 | 20 |
| 1 | 30 | 40 |

The `axis` parameter

## When axis=0 or 'index': (match by row)

The multiplication happens row-wise.

The values from the Series are matched to the rows (index) of the DataFrame.

Example: Each row of the DataFrame is multiplied by the corresponding value from the Series, based on the row index.

df:

|   | A | B |
|---|---|---|
| x | 1 | 4 |
| y | 2 | 5 |
| z | 3 | 6 |

series:

```
x    10
y   100
z  1000
dtype: int64
```



output:

|   | A    | B    |
|---|------|------|
| x | 10   | 40   |
| y | 200  | 500  |
| z | 3000 | 6000 |

Code snippet:

```
df.mul(series_row, axis=0)
or
df.mul(series_row, axis='index')
```

Code Snippet:

```
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
```

df

```
series_row = pd.Series([10, 100, 1000],
                        index=['x', 'y', 'z'])
```

series\_row

df:

|   | A | B |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |



```
x    10
y   100
z  1000
dtype: int64
```

## When axis=1 or axis='columns' (match by column)

The multiplication happens column-wise.

The values from the Series (or similar) are matched to the columns of the DataFrame.

Example: Each column of the DataFrame is multiplied by the corresponding value from the Series, based on the column names.

|     |   |   |         |     |              |         |    |     |
|-----|---|---|---------|-----|--------------|---------|----|-----|
| df: |   |   | series: |     |              | output: |    |     |
|     | A | B |         | A   | B            |         | A  | B   |
| x   | 1 | 4 | A       | 10  |              | x       | 10 | 400 |
| y   | 2 | 5 | B       | 100 |              | y       | 20 | 500 |
| z   | 3 | 6 |         |     | dtype: int64 | z       | 30 | 600 |

Code snippet:

```
df.mul(series_col, axis=1)
```

or

```
df.mul(series_col, axis='columns')
```

Code Snippet:

```
df = pd.DataFrame({  
    'A': [1, 2, 3],  
    'B': [4, 5, 6]  
}, index=['x', 'y', 'z'])
```

df

```
series_col = pd.Series({'A': 10, 'B': 100})  
series_col
```

|     |   |   |         |     |              |
|-----|---|---|---------|-----|--------------|
| df: |   |   | series: |     |              |
|     | A | B |         | A   | B            |
| x   | 1 | 4 | A       | 10  |              |
| y   | 2 | 5 | B       | 100 |              |
| z   | 3 | 6 |         |     | dtype: int64 |



## The `fill_value` parameter

It accepts float or None,  
default None.

It is used to Fill existing missing  
(NaN) values, and any new  
element needed for successful  
DataFrame alignment, with this  
value before computation.

If data in both corresponding  
DataFrame locations is missing  
the result will be missing.

Without fill\_value:

df1:

|   | A   | B   |
|---|-----|-----|
| 0 | 1.0 | 4.0 |
| 1 | 2.0 | NaN |
| 2 | NaN | 6.0 |

df2:

|   | A    | B    |
|---|------|------|
| 0 | NaN  | 40.0 |
| 1 | 20.0 | 50.0 |
| 2 | 30.0 | NaN  |

df1.mul(df2)

or

df1.mul(df2, fill\_value=None)

|   | A    | B     |
|---|------|-------|
| 0 | NaN  | 160.0 |
| 1 | 40.0 | NaN   |

Using fill\_value:

df1:

|   | A   | B   |
|---|-----|-----|
| 0 | 1.0 | 4.0 |
| 1 | 2.0 | NaN |
| 2 | NaN | 6.0 |

df2:

|   | A    | B    |
|---|------|------|
| 0 | NaN  | 40.0 |
| 1 | 20.0 | 50.0 |
| 2 | 30.0 | NaN  |

df1.mul(df2, fill\_value=0)

|   | A    | B     |
|---|------|-------|
| 0 | 0.0  | 160.0 |
| 1 | 40.0 | 0.0   |
| 2 | 0.0  | 0.0   |

|   | A   | B   |
|---|-----|-----|
| 2 | NaN | NaN |

## Sources & References

[pandas.DataFrame.mul Documentation](#)

( <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.mul.html> )

## Other E-books on pandas

[Download](#) ( [https://github.com/kicherethedatascientist/My\\_e-Books](https://github.com/kicherethedatascientist/My_e-Books) )

## Author Biography

Kichere Magubu is a data enthusiast and content creator.

## Contacts and Social Media

**Kichere Magubu**

 Dar es salaam, Tanzania

 [Kichere Magubu](#)

 Kichere The Data Scientist

 KichereTheDataScientist

 KichereTheDataScientist

 kicherethedatascientist@gmail.com

 kicherethedatascientist

 +255 654 729 851

Powered by  
[Eastern Africa Statistical Training Centre](#)

\*\*\*\*\*

## Thank You!

Thank you for reading this e-book! If you found it valuable, please consider leaving an honest review. Your feedback and support mean a lot to me!

```
#print("The cell to convert jupyter notebook to html")  
!jupyter nbconvert --to hide_code_html "df.mul.ipynb"
```